

# Deep-dive struttura simulatore

## psd\_gazebo\_sim/

Questo è probabilmente il cuore di tutta la repo, ovvero la cartella/repo che contiene tutta la descrizione del simulatore, della pista e del veicolo.

La sua struttura è così divisa:

```
psd_gazebo_sim/
├── custom_teleop
│   ├── CMakeLists.txt
│   ├── LICENSE
│   ├── package.xml
│   └── src
├── gz_ros2_control
│   ├── doc
│   ├── Dockerfile
│   ├── gz_ros2_control
│   ├── gz_ros2_control_demos
│   ├── gz_ros2_control_tests
│   ├── LICENSE
│   └── README.md
├── psd_gazebo_worlds
│   ├── CMakeLists.txt
│   ├── package.xml
│   └── world
├── psd_vehicle_bringup
│   ├── CMakeLists.txt
│   ├── config
│   ├── launch
│   ├── LICENSE
│   └── package.xml
├── psd_vehicle_description
│   ├── CMakeLists.txt
│   ├── launch
│   ├── meshes
│   ├── package.xml
│   ├── rviz
│   └── urdf
└── ros_components_description
    ├── CMakeLists.txt
    ├── config
    └── env-hooks
```

```

├─ launch
├─ LICENSE_APACHE2.txt
├─ LICENSE_MIT.txt
├─ LICENSE.txt
├─ meshes
├─ package.xml
├─ README.md
├─ test
└─ urdf

```

Di tutti questi nodi, i nodi evidenziati di **giallo** sono i nodi clonati da repo esistenti che ci permettono di aggiungere feature al simulatore, mentre quelli **verdi** sono la parte "custom" che andiamo di seguito ad espandere:

```

psd_gazebo_sim/
├─ custom_teleop
│   ├── CMakeLists.txt
│   ├── LICENSE
│   ├── package.xml
│   └─ src
│       ├── custom_teleop_gamepad.cpp
│       ├── custom_teleop_keyboard.cpp
│       └─ custom_teleop_logitech.cpp
├─ psd_gazebo_worlds
│   ├── CMakeLists.txt
│   ├── package.xml
│   └─ world
│       ├── models
│       │   ├── cone
│       │   └─ indy_track
│       ├── points_decimated.txt
│       ├── rpy2normal.py
│       ├── track_cones.sdf
│       ├── track_creator.py
│       ├── track_creator_WIP.py
│       ├── track_csv
│       │   ├── track_inner.csv
│       │   └─ track_outer.csv
│       ├── track_decimate.sdf
│       ├── track_develop.sdf
│       ├── track_indy.sdf
│       ├── track_no_collision.sdf
│       └─ track.sdf
├─ psd_vehicle_bringup
│   └─ CMakeLists.txt

```



```

|   └─ visual
|       ├── base_link.dae
|       ├── base_link.stl
|       ├── link_fl_1.dae
|       ├── link_FL_1.stl
|       ├── link_fr_1.dae
|       ├── link_FR_1.stl
|       ├── link_rl_1.dae
|       ├── link_RL_1.stl
|       ├── link_rr_1.dae
|       ├── link_RR_1.stl
|       ├── perno_fl_1.dae
|       ├── perno_FL_1.stl
|       ├── perno_fr_1.dae
|       ├── perno_FR_1.stl
|       ├── tyre_fl_1.dae
|       ├── tyre_FL_1.stl
|       ├── tyre_fr_1.dae
|       ├── tyre_FR_1.stl
|       ├── tyre_rl_1.dae
|       ├── tyre_RL_1.stl
|       ├── tyre_rr_1.dae
|       └─ tyre_RR_1.stl
└─ package.xml
└─ rviz
    └─ psd_vehicle.rviz
└─ urdf
    ├── components
    │   └─ camera_mount.urdf.xacro
    ├── imu.xacro
    ├── materials.xacro
    ├── psd_vehicle_macro.ros2_control.xacro
    ├── psd_vehicle_macro.urdf.xacro
    └─ psd_vehicle.urdf.xacro

```

Andando in ordine dall'alto verso il basso troviamo:

## 1) custom\_teleop/

Contiene i vari nodi per controllare il veicolo simulato mediante tastiera, gamepad e sterzo/pedaliera.

**Subscriber:** —

**Publisher:** steer\_topic ( /position\_controller/commands ), drive\_topic ( /effort\_controller/commands )

## 2) psd\_gazebo\_worlds/

Contiene la descrizione degli "ambienti" dove il modello si muoverà.

Al suo interno infatti troviamo:

```
psd_gazebo_worlds/
├─ CMakeLists.txt
├─ package.xml
├─ world
│   └─ models
│       ├── cone
│       │   ├── meshes
│       │   │   ├── cone.dae
│       │   │   └─ cone.stl
│       │   └─ model.config
│       │       └─ model.sdf
│       └─ indy_track
│           └─ meshes
│               ├── pista_con_muretto.stl
│               ├── pista_inclinata_lineare.stl
│               └─ Pista.stl
├─ points_decimated.txt
├─ rpy2normal.py
├─ track_cones.sdf
├─ track_creator.py
├─ track_creator_WIP.py
├─ track_csv
│   ├── track_inner.csv
│   └─ track_outer.csv
├─ track_decimate.sdf
├─ track_develop.sdf
├─ track_indy.sdf
├─ track_no_collision.sdf
└─ track.sdf
```

Ogni ambiente è descritto in formato **sdf** e si presenta così: (e.g. **track.sdf**)

```
<?xml version="1.0"?>

<sdf version='1.9'>
  <world name='track'>

    <!-- PHYSICS -->
    <physics name="1ms" type="ignored">
```

```

        <max_step_size>0.001</max_step_size>
        <real_time_factor>1.0</real_time_factor>
    </physics>

    <gravity>0 0 -9.8</gravity>
    <magnetic_field>6e-06 2.3e-05 -4.2e-05</magnetic_field>
    <atmosphere type='adiabatic' />

    <!-- ..... -->

    <!-- PLUGINS -->
    <plugin name='gz::sim::systems::Physics' filename='gz-sim-physi
    <plugin name='gz::sim::systems::UserCommands' filename='gz-sim-
    <plugin name='gz::sim::systems::SceneBroadcaster' filename='gz-
    <plugin name="gz::sim::systems::Imu" filename="gz-sim-imu-syste

    <gui fullscreen='0'>
        <!-- 3D scene -->
        <plugin filename="MinimalScene" name="3D View">
            <gz-gui>
                <title>3D View</title>
                <property type="bool" key="showTitleBar">false</property>
                <property type="string" key="state">docked</property>
            </gz-gui>

            <engine>ogre2</engine>
            <scene>scene</scene>
            <ambient_light>0.4 0.4 0.4</ambient_light>
            <background_color>0.8 0.8 0.8</background_color>
            <camera_pose>-6 0 6 0 0.5 0</camera_pose>
        </plugin>

        <!-- Plugins that add functionality to the scene -->
        <plugin filename="EntityContextMenuPlugin" name="Entity con
            <gz-gui>
                <property key="state" type="string">floating</property>
                <property key="width" type="double">5</property>
                <property key="height" type="double">5</property>
                <property key="showTitleBar" type="bool">false</property>
            </gz-gui>
        </plugin>
        <plugin filename="GzSceneManager" name="Scene Manager">
            <gz-gui>
                <property key="resizable" type="bool">false</property>
                <property key="width" type="double">5</property>

```

```

    <property key="height" type="double">5</property>
    <property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
  </gz-gui>
</plugin>
<plugin filename="InteractiveViewControl" name="Interactive
  <gz-gui>
    <property key="resizable" type="bool">>false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property>
    <property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
  </gz-gui>
</plugin>
<plugin filename="CameraTracking" name="Camera Tracking">
  <gz-gui>
    <property key="resizable" type="bool">>false</property>
    <property key="width" type="double">5</property>
    <property key="height" type="double">5</property>
    <property key="state" type="string">floating</property>
    <property key="showTitleBar" type="bool">>false</property>
  </gz-gui>
</plugin>
<plugin name='World control' filename='WorldControl'>
  <gz-gui>
    <title>World control</title>
    <property type='bool' key='showTitleBar'>0</property>
    <property type='bool' key='resizable'>0</property>
    <property type='double' key='height'>72</property>
    <property type='double' key='z'>1</property>
    <property type='string' key='state'>floating</property>
    <anchors target='3D View'>
      <line own='left' target='left' />
      <line own='bottom' target='bottom' />
    </anchors>
  </gz-gui>
  <play_pause>1</play_pause>
  <step>1</step>
  <start_paused>1</start_paused>
</plugin>
<plugin name='World stats' filename='WorldStats'>
  <gz-gui>
    <title>World stats</title>
    <property type='bool' key='showTitleBar'>0</property>
    <property type='bool' key='resizable'>0</property>

```

```

        <property type='double' key='height'>110</property>
        <property type='double' key='width'>290</property>
        <property type='double' key='z'>1</property>
        <property type='string' key='state'>floating</prope
        <anchors target='3D View'>
            <line own='right' target='right' />
            <line own='bottom' target='bottom' />
        </anchors>
    </gz-gui>
    <sim_time>1</sim_time>
    <real_time>1</real_time>
    <real_time_factor>1</real_time_factor>
    <iterations>1</iterations>
</plugin>

<plugin filename="VisualizeLidar" name="Visualize Lidar">
</plugin>

<plugin filename="ImageDisplay" name="Image Display">
</plugin>

<!-- Inspector -->
<plugin filename="ComponentInspector" name="Component inspe
    <ignition-gui>
        <property type="string" key="state">docked</property>
    </ignition-gui>
</plugin>

<!-- Entity tree -->
<plugin filename="EntityTree" name="Entity tree">
    <ignition-gui>
        <property type="string" key="state">docked</property>
    </ignition-gui>
</plugin>
</gui>
<!-- ..... -->

<!-- SENSORS -->

<plugin
filename="gz-sim-sensors-system"
name="gz::sim::systems::Sensors">
<render_engine>ogre2</render_engine>
</plugin>

```



```

<!-- ..... -->

<!-- AMBIENT -->

<scene>
  <ambient>1 1 1 1</ambient>
  <background>0.8 0.8 0.8 1</background>
  <shadows>1</shadows>
</scene>
<light name='sun' type='directional'>
  <cast_shadows>1</cast_shadows>
  <pose>0 0 10 0 0 0</pose>
  <diffuse>0.8 0.8 0.8 1</diffuse>
  <specular>0.8 0.8 0.8 1</specular>
  <attenuation>
    <range>1000</range>
    <constant>0.9</constant>
    <linear>0.01</linear>
    <quadratic>0.001</quadratic>
  </attenuation>
  <direction>-0.5 0.1 -0.9</direction>
</light>

<model name='ground_plane'>
  <static>true</static>
  <link name='link'>
    <collision name='collision'>
      <geometry>
        <plane>
          <normal>0 0 1</normal>
          <size>100 100</size>
        </plane>
      </geometry>

      <surface>
        <friction>
          <dart/>
        </friction>
        <bounce/>
        <contact/>
      </surface>
    </collision>

    <visual name='visual'>

```

```

        <geometry>
            <plane>
                <normal>0 0 1</normal>
                <size>100 100</size>
            </plane>
        </geometry>
        <material>
            <ambient>0.1 0.1 0.1 1</ambient>
            <diffuse>0.1 0.1 0.1 1</diffuse>
            <specular>0.8 0.8 0.8 1</specular>
        </material>
    </visual>

    <pose>0 0 0 0 -0 0</pose>

    <inertial>
        <pose>0 0 0 0 -0 0</pose>
        <mass>1</mass>
        <inertia>
            <ixx>1</ixx>
            <ixy>0</ixy>
            <ixz>0</ixz>
            <iyy>1</iyy>
            <iyz>0</iyz>
            <izz>1</izz>
        </inertia>
    </inertial>

    <enable_wind>false</enable_wind>

</link>

    <pose>0 0 0 0 -0 0</pose>

    <self_collide>false</self_collide>
</model>

<!-- ..... -->

<!-- TRACK -->
<model name='cones'>
    <pose>0 0 0 0 0 0</pose>

    <link name='cone_0_link'>

```

```

        <collision name='cone_0_collision'>
            <geometry>
                <mesh>
                    <uri>/home/ubuntu/psd_ws/src/psd_gazebo_sim/p
                    <scale>0.001 0.001 0.001</scale>
                </mesh>
            </geometry>
        </collision>
        <visual name='cone_0_visual'>
            <geometry>
                <mesh>
                    <uri>/home/ubuntu/psd_ws/src/psd_gazebo_sim/p
                    <scale>0.001 0.001 0.001</scale>
                </mesh>
            </geometry>
            <material>
                <ambient>1 1 0 1</ambient>
                <diffuse>1 1 0 1</diffuse>
                <specular>1 1 0 1</specular>
            </material>
        </visual>
        <pose>5.688073798552464 25.575976039184496 0.0 1.57
        <enable_wind>>false</enable_wind>
    </link>

    <!-- ....Other cones as links.... -->

    <static>true</static>
    <self_collide>>false</self_collide>
</model>
</world>
</sdf>

```

Andando a fondo nella descrizione del file troviamo, a partire dall'inizio:

#### 1. La definizione del motore fisico:

```

<!-- PHYSICS -->
<physics name="1ms" type="ignored">
    <max_step_size>0.001</max_step_size>
    <real_time_factor>1.0</real_time_factor>
</physics>

<gravity>0 0 -9.8</gravity>

```

```
<magnetic_field>6e-06 2.3e-05 -4.2e-05</magnetic_field>
<atmosphere type='adiabatic' />
```

2. La chiamata a tutti i plugin necessari:

```
<!-- PLUGINS -->
<plugin name='gz::sim::systems::Physics' filename='gz-sim-physics-sy
<plugin name='gz::sim::systems::UserCommands' filename='gz-sim-user-
<plugin name='gz::sim::systems::SceneBroadcaster' filename='gz-sim-s
<plugin name="gz::sim::systems::Imu" filename="gz-sim-imu-system"/>
```

3. La definizione della GUI di Gazebo, in particolare:

a. Visualizzazione sensori:

```
<plugin filename="VisualizeLidar" name="Visualize Lidar">
</plugin>

<plugin filename="ImageDisplay" name="Image Display">
</plugin>
```

b. Il motore di rendering della scena con le relative impostazioni di luce:

```
<plugin filename="MinimalScene" name="3D View">
  <gz-gui>
    <title>3D View</title>
    <property type="bool" key="showTitleBar">false</property>
    <property type="string" key="state">docked</property>
  </gz-gui>

  <engine>ogre2</engine>
  <scene>scene</scene>
  <ambient_light>0.4 0.4 0.4</ambient_light>
  <background_color>0.8 0.8 0.8</background_color>
  <camera_pose>-6 0 6 0 0.5 0</camera_pose>
</plugin>
```

4. Motore di rendering dei sensori:

```
<plugin
  filename="gz-sim-sensors-system"
  name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>
```

5. La parte di ambiente, ovvero la parte "realistica":

- a. Illuminazione scena: [consiglio di settare `<shadows>` e `<cast_shadows>` a 0 (False) qualora si volesse massimizzare la reattività del simulatore a discapito del fotorealismo]

```
<scene>
  <ambient>1 1 1 1</ambient>
  <background>0.8 0.8 0.8 1</background>
  <shadows>0</shadows>
</scene>
<light name='sun' type='directional'>
  <cast_shadows>0</cast_shadows>
  <pose>0 0 10 0 0 0</pose>
  <diffuse>0.8 0.8 0.8 1</diffuse>
  <specular>0.8 0.8 0.8 1</specular>
  <attenuation>
    <range>1000</range>
    <constant>0.9</constant>
    <linear>0.01</linear>
    <quadratic>0.001</quadratic>
  </attenuation>
  <direction>-0.5 0.1 -0.9</direction>
</light>
```

b. Il piano di base del mondo (ground\_plane) di cui possiamo impostare:

- i. Inclinazione andando a variare le componenti del vettore normale al piano e il tipo di risolutore per il calcolo dell'attrito e del tipo di contatto:

```
<collision name='collision'>
  <geometry>
    <plane>
      <normal>0 0 1</normal>
      <size>100 100</size>
    </plane>
  </geometry>

  <surface>
    <friction>
      <dart/>
    </friction>
    <contact/>
  </surface>
</collision>
```

ii. Il colore e l'effetto sotto l'illuminazione:

```
<visual name='visual'>
  <geometry>
    <plane>
      <normal>0 0 1</normal>
      <size>100 100</size>
    </plane>
  </geometry>
  <material>
    <ambient>0.1 0.1 0.1 1</ambient>
    <diffuse>0.1 0.1 0.1 1</diffuse>
    <specular>0.8 0.8 0.8 1</specular>
  </material>
</visual>
```

iii. La posa `<pose>0 0 0 0 -0 0</pose>` che rappresenta rispettivamente `x y z roll pitch yaw`

c. La pista infine è definita come un modello unico composto da N link con N pari al numero di coni: questa scelta è stata fatta per poter orientare e spostare la pista in maniera solidale senza andare a calcolare la nuova posa di ogni cono a mano (abbiamo circa 300 coni/pista):

i. Infatti definiamo il modello `<model name='cones'>` definito da una posa `<pose>0 0 0 0 0 0</pose>` a cui sottostanno tutti i link.

ii. Ogni link "cono" contiene una definizione della zona di collisione e della parte visuale. Dunque ci sono due opzioni:

1. La parte `visual` deve contenere la mesh del cono e una descrizione del proprio colore

2. Invece la parte `collision` può contenere:

a. la mesh per maggiore realismo nei contatti e nei limiti di pista

b. un solido semplice (cilindro, parallelepipedo o sfera) per massimizzare le prestazioni, riducendo la complessità dei calcoli

iii. Inoltre il modello complessivo è definito come `static` ovvero che i link non si possono muovere così da ridurre ancora la complessità del modello.

Per la realizzazione della pista, è stato realizzato uno script apposito in python (**track\_creator.py**) che si occupa di generare il file .sdf a partire da una lista di coni interni alla pista (gialli) ed esterni (blu).

```
import numpy as np

def generate_sdf(inner, outer):
```

```

sdf_template = '''<?xml version="1.0"?>

<sdf version='1.9'>
  <world name='track'>

    <!-- PHYSICS -->

    ... omissis in questa guida ...
    {qui ci sono tutte le definizioni di
    plugin e co viste in precedenza che
    sono statiche di pista in pista}

    <!-- TRACK -->

    <!-- TRACK -->
    <model name='cones'>
      <pose>0 0 0 0 0 0</pose>
      {}
      <static>true</static>
      <self_collide>>false</self_collide>
    </model>
  </world>
</sdf>

'''

cone_model_template = '''

<link name='cone_{}_link'>

  <visual name='cone_{}_visual'>
    <geometry>
      <mesh>
        <uri>/home/ubuntu/psd_ws/src/psd_gazebo_sim/psd_gazebo_wo
        <scale>0.001 0.001 0.001</scale>
      </mesh>
    </geometry>
    <material>
      <ambient>{}</ambient>
      <diffuse>{}</diffuse>
      <specular>{}</specular>
    </material>
  </visual>
  <pose>{}</pose>
  <enable_wind>>false</enable_wind>

```

```

</link>'''

cones = ""
for i, coord in enumerate(inner):
    x, y, z, color = coord[0:4]

    pose = "{} {} {} 1.57 0 0".format(x, y ,z)

    #inner = yellow cones
    ambient = '1 1 0 1'
    diffuse = '1 1 0 1'
    specular = '1 1 0 1'

    cones += cone_model_template.format(i, i, ambient, diffuse, spe

for i, coord in enumerate(outer):
    bias = len(inner)
    x, y, z, color = coord[0:4]

    pose = "{} {} {} 1.57 0 0".format(x, y ,z)

    #outer = blue cones
    ambient = '0 0 1 1'
    diffuse = '0 0 1 1'
    specular = '0 0 1 1'

    idx = i + bias + 1
    cones += cone_model_template.format(idx, idx, ambient, diffuse,
return sdf_template.format(cones)

def scale_track(track_coordinates, scale_factor):
    xy_coords = track_coordinates[:, :2]

    centroid = np.mean(xy_coords, axis=0)

    # Translate the track coordinates so that the centroid is at the or
    translated_track = xy_coords - centroid

    # Scale down the translated track uniformly
    scaled_track_xy = translated_track * scale_factor

    # Translate the scaled track back to its original position
    scaled_track_xy += centroid

    scaled_track = np.hstack((scaled_track_xy, track_coordinates[:, 2:4

```



```

        return scaled_track

def main():
    # Assuming the format is: x, y, z, 1 for each cone
    innerConePosition = np.loadtxt("track_csv/track_inner.csv", delimiter=',')
    outerConePosition = np.loadtxt("track_csv/track_outer.csv", delimiter=',')

    scale_factor = 0.5
    scaled_inner = scale_track(innerConePosition, scale_factor)
    scaled_outer = scale_track(outerConePosition, scale_factor)

    # sdf_content = generate_sdf(scaled_inner, scaled_outer)
    sdf_content = generate_sdf(scaled_inner, scaled_outer)

    with open("track_full.sdf", "w") as f:
        f.write(sdf_content)

if __name__ == "__main__":
    main()

```

### 3) psd\_vehicle\_description/

Contiene una descrizione di tutto il veicolo come URDF (link, joint) e delle interfacce attuabili (sterzo, ruote) e sensibili (lidar, imu, camera).

La directory è così strutturata:

```

psd_vehicle_description/
├─ launch
│   └─ view_psd_vehicle.launch.py
├─ meshes
│   └─ collision
│       ├── base_link.stl
│       ├── link_FL_1.stl
│       ├── link_FR_1.stl
│       ├── link_RL_1.stl
│       ├── link_RR_1.stl
│       ├── perno_FL_1.stl
│       ├── perno_FR_1.stl
│       ├── tyre_FL_1.stl
│       ├── tyre_FR_1.stl
│       └── tyre_RL_1.stl

```

```

|   |   └─ tyre_RR_1.stl
|   └─ components
|       ├── camera_mount_bot.dae
|       ├── camera_mount_mid.dae
|       └─ camera_mount_top.dae
|   └─ obj+mtl
|       ├── chassis.mtl
|       ├── chassis.obj
|       ├── steering_hinge.mtl
|       ├── steering_hinge.obj
|       ├── suspension.mtl
|       ├── suspension.obj
|       ├── tyre.mtl
|       ├── tyre.obj
|       ├── wheel.mtl
|       └─ wheel.obj
|   └─ stl
|       ├── chassis.stl
|       ├── steering_hinge.stl
|       ├── suspension.stl
|       ├── tyre.stl
|       └─ wheel.stl
|   └─ visual
|       ├── base_link.dae
|       ├── base_link.stl
|       ├── link_fl_1.dae
|       ├── link_FL_1.stl
|       ├── link_fr_1.dae
|       ├── link_FR_1.stl
|       ├── link_rl_1.dae
|       ├── link_RL_1.stl
|       ├── link_rr_1.dae
|       ├── link_RR_1.stl
|       ├── perno_fl_1.dae
|       ├── perno_FL_1.stl
|       ├── perno_fr_1.dae
|       ├── perno_FR_1.stl
|       ├── tyre_fl_1.dae
|       ├── tyre_FL_1.stl
|       ├── tyre_fr_1.dae
|       ├── tyre_FR_1.stl
|       ├── tyre_rl_1.dae
|       ├── tyre_RL_1.stl
|       ├── tyre_rr_1.dae
|       └─ tyre_RR_1.stl

```

```

├─ urdf
│   ├── components
│   │   └─ camera_mount.urdf.xacro
│   ├── imu.xacro
│   ├── materials.xacro
│   ├── psd_vehicle_macro.ros2_control.xacro
│   ├── psd_vehicle_macro.urdf.xacro
│   └─ psd_vehicle.urdf.xacro
├─ rviz
│   └─ psd_vehicle.rviz
├─ package.xml
└─ CMakeLists.txt

```

Possiamo vedere che la struttura in questo caso è ben delineata con la cartella `meshes/` dedicata a contenere tutte le mesh nei vari formati (stl, obj, dae) e infine la cartella `urdf/` con tutta la parte descrittiva del veicolo.

La gerarchia dei file per la descrizione dei robot è la seguente:

1. `psd_vehicle.urdf.xacro` è una sorta di "launcher" in cui vengono importati i file per la definizione fisica del veicolo, del controllo, dei sensori e in cui viene definita la posa di spawn del robot.

```

<?xml version="1.0" encoding="UTF-8"?>
<robot xmlns:xacro="http://wiki.ros.org/xacro" name="psd_vehicle">

  <xacro:include filename="$(find psd_vehicle_description)/urdf/psd_
  <xacro:include filename="$(find psd_vehicle_description)/urdf/psd_

  <xacro:property name="PI" value="3.14159265359" />
  <xacro:property name="rot90" value="{90 * PI / 180}" />

  <!-- Load robot's macro with parameters -->
  <link name="home" />

  <!-- set prefix if multiple robots are used -->
  <xacro:psd_vehicle parent="home" >
    <origin xyz="0 0 0.095" rpy="0 0 -${rot90}" />      <!-- pos
  </xacro:psd_vehicle>

  <xacro:psd_vehicle_ros2_control
    name="psd_vehicle"
  />

  <xacro:include filename="$(find psd_vehicle_description)/urdf/imu.

```

```
</robot>
```

2. `psd_vehicle_macro.urdf.xacro` : qui viene definito proprio l'albero che lega tutti i vari componenti del veicolo tra loro, dandone anche una descrizione fisica in termini di massa, matrice di inerzia e materiale.

Per approfondimenti su questa parte, si rimanda alla presentazione powerpoint allegata.

3. `psd_vehicle_macro.ros2_control.xacro` : qui invece sono definite le interfacce attuate e anche la IMU

Per approfondimenti su questa parte, si rimanda alla presentazione powerpoint allegata.

4. `materials.xacro` : raccoglie alcune definizioni di materiali costituenti il robot, per i quali sono definiti colore, opacità e anche, nel caso delle gomme, le proprietà fisiche di interazione con il pavimento (attrito, elasticità, ...)
5. `imu.xacro` : contiene le definizioni dei vari rumori sulle varie grandezze misurate.

Gli altri sensori, ovvero lidar e camera (ZED2) sono stati importati da una repo esistente mantenuta dalla Husarion, denominata `ros_components`. In questa sono state infatti replicate in maniera fedele le proprietà fisiche e di misura di questi sensori.

## Sensori

### 1) LiDAR (`psd_vehicle_macro.urdf.xacro`)

```
<!-- SENSORS -->
<!-- lidar -->
<xacro:include filename="$(find ros_components_description)/urdf/sl

<!-- evaluate the macro and place the sensor on robot -->
<xacro:lidar.slamtec_rplidar_s1
  parent_link="chassis"
  xyz="0.0 0.0 0.1"
  rpy="0.0 0.0 0.0" />
```

Qui viene definito il collocamento del lidar sul veicolo, ma la definizione del sensore vero e proprio si trova nel file incluso da `ros_components/urdf`

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:macro name="slamtec_rplidar_s1"
    params="parent_link xyz rpy
      namespace:=None
      device_namespace:=None ">

    // ----- omissis ----- //
    qui viene descritta la parte di modello
      e grafica del sensore
    // ----- omissis ----- //

    <gazebo reference="${prefix}laser">
      <!-- gpu_lidar has to be set, CPU lidar doesn't work in ignition
      https://github.com/gazebo/gz-sensors/issues/26 -->
      <sensor type="gpu_lidar" name="${ns}${prefix}slamtec_rplidar_s1_s

        <topic>${ns}${device_ns}scan</topic>
        <frame_id>${ns}${prefix}laser</frame_id>
        <ignition_frame_id>${ns}${prefix}laser</ignition_frame_id>

        <update_rate>10.0</update_rate>
        <ray>
          <scan>
            <horizontal>
              <samples>920</samples>
              <resolution>1</resolution>
              <min_angle>-${pi}</min_angle>
              <max_angle>${pi}</max_angle>
            </horizontal>
          </scan>
          <range>
            <min>0.04</min>
            <max>40.0</max>
            <resolution>0.03</resolution>
          </range>
          <noise>
            <type>gaussian</type>
            <mean>0.0</mean>
            <stddev>0.001</stddev>
          </noise>
        </ray>
        <always_on>true</always_on>
        <visualize>>false</visualize>

```

```

    <ros>
      <namespace>${ns}${device_ns}</namespace>
    </ros>
  </sensor>
</gazebo>

</xacro:macro>
</robot>

```

In particolare in quest'ultima parte troviamo la definizione delle caratteristiche del sensore con il range, rumore, update rate e tutto il necessario per impostare il sensore come desideriamo.

## 2) Stereocamera (*psd\_vehicle\_macro.urdf.xacro*)

```

<!-- stereocamera -->
<xacro:include filename="$(find ros_components_description)/urdf/in

<!-- evaluate the macro and place the sensor on robot -->
<xacro:camera.intel_realsense_d435
  parent_link="chassis"
  xyz="0.0 0.0 0.2"
  rpy="0.0 0.0 ${rot90}" />

<!-- position/odom -->
<link name="base_footprint"/>

<joint name="footprint_joint" type="fixed">
  <parent link="chassis"/>
  <child link="base_footprint"/>
  <origin xyz="0.0 0.0 0.0" rpy="0 0 0"/>
</joint>

```

Qui viene definito il collocamento della stereocamera sul veicolo, ma la definizione del sensore vero e proprio si trova nel file incluso da `ros_components/urdf`

```

<?xml version="1.0"?>
<robot name="intel_realsense_d435" xmlns:xacro="http://ros.org/wiki/xac
  <xacro:macro name="intel_realsense_d435"
    params="parent_link xyz rpy
      use_nominal_extrinsics:=false
      namespace:=None
      device_namespace:=camera">

    <xacro:if value="${namespace == 'None'}">

```

```

    <xacro:property name="ns" value="" />
  </xacro:if>
  <xacro:unless value="{namespace == 'None'}">
    <xacro:property name="ns" value="{namespace}/" />
  </xacro:unless>

  <!-- The following values are approximate, and the camera node
    publishing TF values with actual calibrated camera extrinsic values -->
  <xacro:property name="d435_cam_depth_to_infra1_offset" value="0.0" />
  <xacro:property name="d435_cam_depth_to_infra2_offset" value="-0.05" />
  <xacro:property name="d435_cam_depth_to_color_offset" value="0.015" />

  <!-- The following values model the aluminum peripheral case for the
    D435 camera, with the camera joint represented by the actual
    peripheral camera tripod mount -->
  <xacro:property name="d435_cam_width" value="0.090" />
  <xacro:property name="d435_cam_height" value="0.025" />
  <xacro:property name="d435_cam_depth" value="0.02505" />
  <xacro:property name="d435_cam_mount_from_center_offset" value="0.0" />
  <!-- glass cover is 0.1 mm inwards from front aluminium plate -->
  <xacro:property name="d435_glass_to_front" value="0.0001" />
  <!-- see datasheet Revision 007, Fig. 4-4 page 65 -->
  <xacro:property name="d435_zero_depth_to_glass" value="0.0042" />
  <!-- convenience precomputation to avoid clutter-->
  <xacro:property name="d435_mesh_x_offset"
    value="{d435_cam_mount_from_center_offset-d435_glass_to_front-d435_zero_depth_to_glass}" />

  <!-- The following offset is relative to the physical D435 camera
    camera tripod mount -->
  <xacro:property name="d435_cam_depth_px" value="{d435_cam_mount_from_center_offset-d435_glass_to_front-d435_zero_depth_to_glass}" />
  <xacro:property name="d435_collision_offset_x" value="0.00824" />
  <xacro:property name="d435_cam_depth_py" value="0.0175" />
  <xacro:property name="d435_cam_depth_pz" value="{d435_cam_height/2}" />

  <!-- camera body, with origin at bottom screw mount -->
  <joint name="{parent_link.rstrip('_link')}_to_{device_namespace}_camera" type="fixed" />
  <origin xyz="{xyz}" rpy="{rpy}" />
  <parent link="{parent_link}" />
  <child link="{device_namespace}_bottom_screw_frame" />
</joint>

<link name="{device_namespace}_bottom_screw_frame" />

<joint name="{device_namespace}_bottom_screw_to_{device_namespace}_camera" type="fixed" />
<origin xyz="{d435_mesh_x_offset} {d435_cam_depth_py} {d435_cam_depth_pz}" />

```

```

    rpy="0.0 0.0 0.0" />
  <parent link="${device_namespace}_bottom_screw_frame" />
  <child link="${device_namespace}_link" />
</joint>

<link name="${device_namespace}_link">
  <visual>
    <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0" />
    <geometry>
      <mesh filename="package://ros_components_description/meshes/i
    </geometry>
  </visual>
  <collision>
    <origin xyz="${d435_collision_offset_x} ${d435_cam_depth_py}
    <geometry>
      <box size="${d435_cam_depth} ${d435_cam_width} ${d435_cam_hei
    </geometry>
  </collision>
  <inertial>
    <!-- The following are not reliable values, and should not be u
    <mass value="0.072" />
    <origin xyz="0.0 0.0 0.0" />
    <inertia ixx="0.003881243" ixy="0.0" ixz="0.0"
      iyy="0.000498940" iyz="0.0"
      izz="0.003879257"

  </inertial>
</link>

<!-- Use the nominal extrinsics between camera frames if the calibr
published. e.g. running the device in simulation -->
<xacro:if value="${use_nominal_extrinsics}">
  <!-- camera depth joints and links -->
  <joint name="${device_namespace}_to_${device_namespace}_depth_joi
    <origin xyz="0.0 0.0 0.0" rpy="0.0 0.0 0.0" />
    <parent link="${device_namespace}_link" />
    <child link="${device_namespace}_depth_frame" />
  </joint>

  <link name="${device_namespace}_depth_frame" />

  <joint name="${device_namespace}_depth_to_${device_namespace}_dep
    <origin xyz="0.0 0.0 0.0" rpy="${-pi/2.0} 0.0 ${-pi/2.0}" />
    <parent link="${device_namespace}_depth_frame" />
    <child link="${device_namespace}_depth_optical_frame" />
  </joint>

```



```

<link name="${device_namespace}_depth_optical_frame" />

<!-- camera left IR joints and links -->
<joint name="${device_namespace}_to_${device_namespace}_infra1_jo
  <origin xyz="0.0 ${d435_cam_depth_to_infra1_offset} 0.0" rpy="0
  <parent link="${device_namespace}_link" />
  <child link="${device_namespace}_infra1_frame" />
</joint>

<link name="${device_namespace}_infra1_frame" />

<joint name="${device_namespace}_infra1_to_${device_namespace}_in
  <origin xyz="0.0 0.0 0.0" rpy="${-pi/2.0} 0.0 ${-pi/2.0}" />
  <parent link="${device_namespace}_infra1_frame" />
  <child link="${device_namespace}_infra1_optical_frame" />
</joint>

<link name="${device_namespace}_infra1_optical_frame" />

<!-- camera right IR joints and links -->
<joint name="${device_namespace}_to_${device_namespace}_infra2_jo
  <origin xyz="0.0 ${d435_cam_depth_to_infra2_offset} 0.0" rpy="0
  <parent link="${device_namespace}_link" />
  <child link="${device_namespace}_infra2_frame" />
</joint>

<link name="${device_namespace}_infra2_frame" />

<joint name="${device_namespace}_infra2_to_${device_namespace}_in
  <origin xyz="0.0 0.0 0.0" rpy="${-pi/2.0} 0.0 ${-pi/2.0}" />
  <parent link="${device_namespace}_infra2_frame" />
  <child link="${device_namespace}_infra2_optical_frame" />
</joint>

<link name="${device_namespace}_infra2_optical_frame" />

<!-- camera color joints and links -->
<joint name="${device_namespace}_to_${device_namespace}_color_joi
  <origin xyz="0.0 ${d435_cam_depth_to_color_offset} 0.0" rpy="0.
  <parent link="${device_namespace}_link" />
  <child link="${device_namespace}_color_frame" />
</joint>

<link name="${device_namespace}_color_frame" />

```

```

<joint name="${device_namespace}_color_to_${device_namespace}_col
  <origin xyz="0.0 0.0 0.0" rpy="${-pi/2.0} 0.0 ${-pi/2.0}" />
  <parent link="${device_namespace}_color_frame" />
  <child link="${device_namespace}_color_optical_frame" />
</joint>

<link name="${device_namespace}_color_optical_frame" />
</xacro:if>

<!-- It is also possible to use single rgb_d_camera sensor, but using
      should be more accurate for D435 - different frames and fovs can
<gazebo reference="${device_namespace}_link">
  <!-- https://github.com/IntelRealSense/realsense-ros#published-to
  <sensor type="camera" name="${ns}${device_namespace}_intel_realse
    <always_on>true</always_on>
    <update_rate>30.0</update_rate>

    <topic>${ns}${device_namespace}/color/image_raw</topic>
    <visualize>>false</visualize>

    <ignition_frame_id>${device_namespace}_color_optical_frame</ign
    <horizontal_fov>${69.0/180.0*pi}</horizontal_fov>
    <image>
      <width>1280</width>
      <height>720</height>
      <format>R8G8B8</format>
    </image>
    <clip>
      <near>0.02</near>
      <far>300.0</far>
    </clip>
    <ros>
      <namespace>${ns}${device_namespace}</namespace>
    </ros>
  </sensor>

  <sensor type="depth_camera" name="${ns}${device_namespace}_intel_
    <always_on>true</always_on>
    <update_rate>30.0</update_rate>

    <topic>${ns}${device_namespace}/depth/image_rect_raw</topic>
    <visualize>>false</visualize>

    <ignition_frame_id>${device_namespace}_depth_optical_frame</ign

```

```

<camera>
  <horizontal_fov>${87.0/180.0*pi}</horizontal_fov>
  <image>
    <width>1280</width>
    <height>720</height>
    <format>R_FLOAT32</format>
  </image>
  <clip>
    <near>0.28</near>
    <far>8.0</far>
  </clip>
  <noise>
    <type>gaussian</type>
    <mean>0.0</mean>
    <stddev>0.005</stddev>
  </noise>
</camera>
<ros>
  <namespace>${ns}${device_namespace}</namespace>
</ros>
</sensor>
</gazebo>
</xacro:macro>
</robot>

```

## 4) psd\_vehicle\_bringup/

Contiene il launcher generale di tutto il simulatore con l'inizializzazione e la descrizione dei controller che gestiscono il controllo dell'attuazione a basso livello.

Questa infatti ha una struttura a livello di cartella molto semplice:

```

psd_vehicle_bringup/
├─ config
│   └─ psd_vehicle_controllers.yaml
├─ launch
│   └─ gz_sim.launch.py
├─ LICENSE
├─ CMakeLists.txt
└─ package.xml

```

Il cuore di questa sezione risiede proprio nel `launch/gz_sim.launch.py` :

1. Inizialmente vengono fatti tutti gli import necessari per farlo essere un `launcher` (infatti vengono richiamate tante funzioni come `FindExecutable` , `Command` , `FindPackageShare` e altri

che servono a richiamare altri file in altre cartelle così da poterli eseguire:

```
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, ExecuteProcess, IncludeLaunchDescription
from launch.actions import RegisterEventHandler
from launch.event_handlers import OnProcessExit
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import Command, FindExecutable, LaunchConfiguration

from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from ament_index_python import get_package_share_directory
```

2. Successivamente troviamo una parte più standardizzata necessaria per definire il sistema di riferimento della camera:

```
# The frame of the point cloud from ignition gazebo 6 isn't provided
# See https://github.com/gazebosim/gz-sensors/issues/239
def fix_depth_image_tf(context, *args, **kwargs):
    robot_namespace = LaunchConfiguration("robot_namespace").perform(context)
    device_namespace = LaunchConfiguration("device_namespace").perform(context)
    tf_prefix = LaunchConfiguration("tf_prefix").perform(context)
    camera_name = device_namespace

    device_namespace_ext = device_namespace + "/"
    if device_namespace == "":
        device_namespace_ext = ""

    tf_prefix_ext = tf_prefix + "_"
    if tf_prefix == "":
        tf_prefix_ext = ""

    parent_frame = tf_prefix_ext + camera_name + "_depth_optical_frame"
    child_frame = (
        "psd_vehicle/chassis/"
        + device_namespace_ext
        + tf_prefix_ext
        + camera_name
        + "_stereolabs_zed_depth"
    )

    static_transform_publisher = Node(
        package="tf2_ros",
```

```

        executable="static_transform_publisher",
        name="point_cloud_tf",
        output="log",
        arguments=["0", "0", "0", "1.57", "-1.57", "0", parent_frame],
        parameters=[{"use_sim_time": True}],
        namespace=robot_namespace,
    )
    return [static_transform_publisher]

```

3. Adesso arriviamo al centro del launcher, ovvero dove vengono messi insieme tutti i blocchi per creare la simulazione vera e propria. Infatti in ordine troviamo:

- a. Viene inizializzato il timer di esecuzione per usare di default quello interno al simulatore

```

def generate_launch_description():
    use_sim_time = LaunchConfiguration('use_sim_time', default=True)

```

- b. Viene inizializzata la stereocamera con il suo tf visto prima e il suo namespace

```

# SENSORS
ros_components_description = get_package_share_directory("ros_gz_bridge")
gz_bridge_config_path = os.path.join(
    ros_components_description, "config", "gz_stereolabs_zed2.yaml"
)

declare_device_namespace = DeclareLaunchArgument(
    "device_namespace",
    default_value="zed2",
    description="Sensor namespace that will appear before all links"
)

declare_tf_prefix = DeclareLaunchArgument(
    "tf_prefix",
    default_value="",
    description="Prefix added for all links of device. Here u

```

- c. Il namespace del robot e il tipo di bridge desiderato (cambia in base alla versione del simulatore usato)

```

declare_robot_namespace = DeclareLaunchArgument(
    "robot_namespace",
    default_value=EnvironmentVariable("ROBOT_NAMESPACE", default="robot")
)

```

```

        description="Namespace which will appear in front of all
    )

    declare_gz_bridge_name = DeclareLaunchArgument(
        "gz_bridge_name",
        default_value="gz_bridge",
        description="Name of gz bridge node.",
    )

```

- d. Viene a questo punto caricata la descrizione del robot (veicolo), andando a convertire i vari urdf definiti per giunti, link, interfacce, materiali e altro, in un'unica descrizione

```

# Get URDF via xacro
robot_description_content = Command(
    [
        PathJoinSubstitution([FindExecutable(name='xacro')]),
        ' ',
        PathJoinSubstitution(
            [FindPackageShare('psd_vehicle_description'),
             'urdf', 'psd_vehicle.urdf.xacro']
        ),
    ]
)
robot_description = {'robot_description': robot_description_c

```

In particolare questa struttura, che ritroveremo anche in seguito, non fa altro che:

1. `Command`: esegue un comando sul terminale
2. `PathJoinSubstitution`: unisco i pezzi con i path
3. `FindPackageShare`: cerco il file che risiede nella cartella così denominata

Dunque tutta la parte dentro le parentesi di command non fa altro che lanciare: `xacro`

`<path_fino_al_package>/psd_vehicle_description/urdf/psd_vehicle.urdf.xacro`

- e. A questo punto, dato che ho il robot, inizializzo prima il publisher dello stato dei giunti di quest'ultimo e poi lo faccio "spawnare" all'interno del frame di Gazebo

```

node_robot_state_publisher = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    output='screen',
    parameters=[robot_description]
)

gz_spawn_entity = Node(
    package="ros_gz_sim",
    executable="create",
    name="spawn_psd_vehicle",
    arguments=[
        "-name",
        "psd_vehicle",
        "-allow_renaming",
        "true",
        "-topic",
        "robot_description",
        "-x",
        "0.0",
        "-y",
        "-1.25",
        "-z",
        "1.0",
    ],
    output="screen",
)

```

- f. Inizializzo i processi che andranno a controllare i giunti mobili definiti nel file `.ros2_control.xacro` in description

```

load_joint_state_broadcaster = ExecuteProcess(
    cmd=['ros2', 'control', 'load_controller', '--set-state',
        'joint_state_broadcaster'],
    output='screen'
)

load_effort_controller = ExecuteProcess(
    cmd=['ros2', 'control', 'load_controller', '--set-state',
        'effort_controller'],
    output='screen'
)

```

```
load_position_controller = ExecuteProcess(
    cmd=['ros2', 'control', 'load_controller', '--set-state',
        'position_controller'],
    output='screen'
)
```

- g. A questo punto, definisco la mappa che voglio utilizzare e la carico nell'ambiente Gazebo

```
world_file = LaunchConfiguration('world_file', default='/home
declare_world_file = DeclareLaunchArgument(
    'world_file',
    default_value=world_file,
    description='Full path to the world file to use in Gazebo
)

gz_sim = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('ros_gz_sim'),
        'launch', 'gz_sim.launch.py')]),
    launch_arguments={'gz_args': [world_file]}.items(),
)
```

- h. Per interfacciare il simulatore gazebo con il nostro framework ROS2, dobbiamo avviare anche un nodo che faccia da ponte tra i due mondi, traducendo i messaggi da un topic sul sim ad uno su ROS2 e viceversa a seconda dei casi (andiamo anche a rimappare qualche nome di topic in una versione più compatta):

```
# Bridge
# https://github.com/gazebosim/ros_gz/blob/ros2/ros_gz_bridge
gz_bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    parameters=[{"config_file": gz_bridge_config_path}],
    arguments=[
        "/clock" + "@rosgraph_msgs/msg/Clock" + "[gz.msgs.Clock]",
        "/scan" + "@sensor_msgs/msg/LaserScan" + "[gz.msgs.LaserScan]",

        # "/velodyne_points/points"
        # + "@sensor_msgs/msg/PointCloud2"
        # + "[gz.msgs.PointCloudPacked]",

        "/camera/color/camera_info"
        + "@sensor_msgs/msg/CameraInfo"
```



```

        + "[gz.msgs.CameraInfo",

        "/camera/color/image_raw"
        + "@sensor_msgs/msg/Image"
        + "[gz.msgs.Image",

        "/camera/camera_info"
        + "@sensor_msgs/msg/CameraInfo"
        + "[gz.msgs.CameraInfo",

        "/camera/depth" + "@sensor_msgs/msg/Image" + "[gz.msg

        "/camera/depth/points"
        + "@sensor_msgs/msg/PointCloud2"
        + "[gz.msgs.PointCloudPacked",

        "/world/track/model/psd_vehicle/link/home/sensor/imu_
        + "@sensor_msgs/msg/Imu"
        + "[gz.msgs.IMU",

    ],

    remappings=[
        ("/velodyne_points/points", "/velodyne_points"),
        ("/camera/camera_info", "/camera/depth/camera_info"),
        ("/camera/depth", "/camera/depth/image_raw"),
        ("/world/track/model/psd_vehicle/link/home/sensor/imu_
    ],

    output='screen'
)

```

i. Infine non resta che lanciare tutte le configurazioni e i comandi predisposti:

```

return LaunchDescription([
    declare_device_namespace,
    declare_robot_namespace,
    declare_tf_prefix,
    declare_gz_bridge_name,
    gz_bridge,
    # Launch gazebo environment
    declare_world_file,
    gz_sim,

```

```

RegisterEventHandler(
    event_handler=OnProcessExit(
        target_action=gz_spawn_entity,
        on_exit=[load_joint_state_broadcaster],
    )
),

RegisterEventHandler(
    event_handler=OnProcessExit(
        target_action=load_joint_state_broadcaster,
        on_exit=[load_effort_controller, load_position_co
    )
),

node_robot_state_publisher,
gz_spawn_entity,
#robot_localization_node,

# Launch Arguments
DeclareLaunchArgument(
    'use_sim_time',
    default_value=use_sim_time,
    description='If true, use simulated clock'),
])

```

4. Ultimo, ma non per importanza, troviamo il file `/config/psd_vehicle_controllers.yaml`, al cui interno sono definiti i plugin e le modalità con cui questi si devono interfacciare con i vari giunti attuti definiti nel file `.ros2_control.xacro` nel `psd_vehicle_description`.

```

controller_manager:
  ros__parameters:
    update_rate: 1000 # Hz

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    effort_controller:
      type: effort_controllers/JointGroupEffortController

    position_controller:
      type: forward_command_controller/ForwardCommandController

```

```
effort_controller:
  ros__parameters:
    joints:
      - drive_fr
      - drive_fl
      - drive_rr
      - drive_rl

position_controller:
  ros__parameters:
    joints:
      - steering_fl
      - steering_fr
    interface_name: position
```